Week 3: Binary search and linked lists

Finding a measurement in ordered records

A device stores sorted measurements in rows. When a new reading arrives, can we find it without inspecting every record? Then a queue of records must be reordered while preserving the same node objects.

This week uses candidate intervals for search and changes links to reverse or merge lists.

Week 3 problems and learning outcomes

Session 5: matrix and rotated-array search. Session 6: list reversal and sorted-list merge.

- Define candidate intervals and node references.
- Explain discarded candidates and preserved nodes.
- Trace updates, prove termination, and analyze cost.

Practice: LeetCode 74, 33, 206, 21. Examples and traces are original teaching examples.

Closed interval and Python indexing

- Candidate indices: `left` through `right`, both included
- Initial bounds: `left=0`, `right=n-1`
- Continue while `left <= right`
- Exclude the checked midpoint with `mid+1` or `mid-1`
- Row/column mapping: integer division `//` and remainder `%`

Search interval: preservation and progress

For sorted `[2,4,7,9]`, target `7`:

Bounds	Middle	Comparison	Next bounds
[0,3]	1	4 < 7	[2,3]
[2,3]	2	7 = 7	found

Invariant: any target occurrence remains in `[left,right]`. Each unsuccessful comparison removes the midpoint and one side. Exit with `left > right`: no candidate remains.

74 · Problem and state

LeetCode 74: Search a 2D Matrix · Medium

Return whether target occurs in the matrix.

`matrix=[[1,4,7],[9,12,18]], target=12 → True`

Keep a closed candidate interval [left,right]; read mid at row mid//C and column mid%C.

Flattening a matrix by index

For three columns, flat index 4 corresponds to row `4 // 3 = 1` and column `4 % 3 = 1`. The matrix remains unchanged and no flattened list is created.

The row order alone is insufficient: the first value of each row must exceed the final value of the preceding row.

State trace

left,right	mid	value	target=12 next interval
0,5	2	7	[3,5]
3,5	4	12	found

Explain the discarded candidates, preserved state, and next position at each row.

Matrix search implementation

```
def search_matrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        if value < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

74 · Correctness and cost

The cross-row property makes the flattened order sorted. Comparisons eliminate an impossible half; every update shrinks the interval, and an empty interval means absence.

R,C are row/column counts. O(log(RC)) time, O(1) auxiliary space; no flattened copy.

33 · Problem and state

LeetCode 33: Search in Rotated Sorted Array · Medium

Return target's index in a rotated sorted array, or -1.

`nums=[6,8,1,3,5], target=3 → 3`

Identify a sorted half and retain it only when its value range contains target.

Rotated array: retain a sorted half

In `[6,8,1,3,5]`, the midpoint is 1. The right half `[1,3,5]` is sorted and contains target 3, so discard the left half and the checked midpoint.

When values are distinct, at least one half is sorted. Compare the target with that half's endpoints before choosing the remaining interval.

Rotated array implementation

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

33 · Correctness and cost

Distinct values and one rotation ensure a sorted half. Range tests discard only an impossible half; mid±1 guarantees progress.

n is length. O(log n) time, O(1) auxiliary space.

Failure inputs and concept checks

left=mid can stall on two candidates. [[1,9],[2,10]] has sorted rows but violates this contract.

Does minimum-feasible search have the same exit meaning?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Session 6 · Node references and link updates

Reorder queued record objects without recreating them. Reuse the existing nodes and change their next references. Trace the order: save the next reference, then reverse the current link.

Node construction and references

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

`ListNode(4)` creates a node. `self.val` stores its value; `self.next` refers to its successor. `None` marks the end.

References and node identity

A node stores a value and a reference to the next node. `a = b` gives two names for the same object; changing `a.next` changes that object's link.

Equal values do not imply identical nodes. `None` marks the end of a singly linked list.

206 · Problem and state

LeetCode 206: Reverse Linked List · Easy

Reverse a singly linked list and return its new head.

`head=[4,6,9] → [9,6,4]`

Save following, redirect current.next to previous, then advance both references.

Reversing one link

For `4 → 6 → 9`, save `following = current.next` before setting `current.next = previous`. Then advance both references.

`previous` starts at `None`; it becomes the new head when `current` reaches `None`. The algorithm reuses the original nodes.

State trace

step	previous	current	saved following
start	None	4→6→9	unset
4 process	4→None	6→9	6
6 process	6→4	9	9
9 process	9→6→4	None	None

Explain how each node moves from the unprocessed suffix to the reversed prefix.

List reversal implementation

```
def reverse_list(head):
    previous, current = None, head
    while current:
        following = current.next
        current.next = previous
        previous, current = current, following
    return previous
```

206 · Correctness and cost

previous holds the reversed prefix and current the untouched suffix. Moving one node preserves all nodes; when current is None, previous is the full reversed list.

n is node count. O(n) time, O(1) auxiliary space; nodes are reused.

21 · Problem and state

LeetCode 21: Merge Two Sorted Lists · Easy

Splice two sorted linked lists into one sorted list.

```
first=[1,5], second=[2,3] → [1,2,3,5]
```

Maintain tail after a dummy node; link the smaller head, then attach the remaining suffix.

Merging sorted lists

For `1 → 5` and `2 → 3`, append the smaller head to the tail: choose 1, then 2, then 3, and attach the remaining 5.

A dummy node makes the first insertion follow the same rule as later insertions. Return `dummy.next`; the dummy is not an input node.

Sorted list merge implementation

```
def merge_lists(first, second):
    dummy = tail = ListNode()
    while first and second:
        if first.val <= second.val:
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first or second
    return dummy.next
```

21 · Correctness and cost

The prefix through tail is sorted. The smaller head is minimal among all remaining nodes; the leftover suffix is already sorted.

n,m are node counts. O(n+m) time, O(1) auxiliary space; input links change.

Failure inputs and concept checks

Overwriting without saving next loses the suffix. Two nodes with equal values are still different objects.

Are two equal-valued nodes one visited object?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Baseline methods and cost

Task	Baseline	Current method
Matrix search	scan: O(RC)	indexed search: O(log(RC))
Rotated search	scan: O(n)	sorted-half search: O(log n)
Reverse list	store nodes: O(n) extra space	relink: O(1) extra space
Merge lists	collect/sort nodes: O(N log N) time	merge: O(N) time

Here N=n+m. Costs assume constant-time integer comparisons and reference operations. Input adapters and test checks allocate extra storage outside the algorithms.

Independent practice and sources

Implement `student_week_03.py` before reading the solutions. Submit state definitions, correctness arguments, cost analysis, and boundary inputs.

Exercises · Worked companion · Instructor guide

Week 3 code: run `python verify.py`; use `--student` to check your implementation.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).